Developer Intern Task: WhatsApp Appointment Booking

Time: about 4 hours of work. Send it back within 4 days of getting this. Use any language and any tools you like, including AI. Be ready to explain every decision.

The situation

A dental clinic with two doctors takes appointments over WhatsApp. Patients write the way they talk: "kal shaam 6 baje", "next Friday", "21/11 at 10:30". Today the receptionist reads every message and writes bookings in a notebook. Mistakes happen: two patients get the same slot, or someone is booked on a day the clinic is closed.

The clinic wants a small booking system where an AI reads the message and the code makes the booking. A wrong booking is worse than asking the patient a question. A patient who comes in at the wrong time, or finds someone else in their slot, is a patient the clinic loses.

What you get

- `clinic.json`: the doctors, their working hours, slot length (30 minutes), closed dates, the timezone, and a fixed `now`. Use this `now` everywhere instead of the real clock, so results can be repeated.
- `messages.json`: 15 WhatsApp messages, each with an `id`, the sender's phone number (`from`) and the `text`. Process them in order. Two of them (`m09` and `m10`) arrive at the same moment.

What to build

A small API (any framework) plus a script that plays the 15 messages through it.

1. **Availability and booking API.** At least: list free slots for a day, book a slot, cancel a booking.
   - The rules live in code: working hours, closed dates, slot length, no bookings in the past.
   - **Two people can never get the same slot**, even when the requests arrive at the same moment. Prove it with a test that sends parallel requests.
   - A patient can only cancel their own booking, identified by their phone number.
2. **Message understanding.** Use an LLM to turn each message into a structured request: intent (book, cancel or other), date, time and doctor, each of which may be missing.
   - Use any LLM API you like. If you have no API key, use a mock or rules, but say so.
   - Validate the AI output strictly. Broken JSON, wrong types, made-up doctors or impossible dates must not crash the run or turn into a booking.
   - Retry in a sensible way, and limit the number of tries.
   - The message text comes from strangers. Nothing in it can change the clinic's rules.
3. **Decide and reply.** For each message, the system either books, cancels, refuses, or asks one clear question. Write the reply the patient would get on WhatsApp. When a request cannot be met, offer real free slots that are close to what they asked for.
4. **Store bookings** somewhere simple (a file or SQLite is fine), so the receptionist can see the day's appointments with a CLI command or a very small page.

5. **Measures itself.** Write down what you think the right outcome is for each of the 15 messages. Then write a script that runs them and prints:
    - wrong bookings (wrong day, time or doctor, or a double booking): the number that matters most,
    - questions asked when the request was clear enough to book,
    - messages handled correctly.

What to send back

- Your code (GitHub link or zip) with a README that lets us run it in under 5 minutes.
- A short note (one page max) answering:
    - Which decisions does the AI make and which does your code make? Why did you split it that way?
    - How do you stop double bookings, and how did you test it?
    - When does the system ask a question instead of booking? Show one message where you chose to ask.
    - What went wrong when you ran it on the 15 messages? Show one real failure.
    - What would you change with another day of time, and what did you not get to?

What we look at

- Does a wrong booking or a double booking ever happen, and do you know when it could?
- Are the clinic's rules enforced in code, not left to the AI?
- Does the system survive bad model output and odd messages (Hindi, Hinglish, different date formats)?
- Can you explain every choice, including the ones AI helped with?
- Honesty. A smaller system that is honest about its limits beats a bigger one that hides them.

We do not need: a real WhatsApp connection, login, deployment, a database server, or a pretty UI.